Lecture 7: Function Approximation in Reinforcement Learning

(And Deep Reinforcement Learning)

Hado van Hasselt
UCL, 2021

Background reading. Sutton & Barto (2018), Chapters 9 + 10 (+ 11).

1 Motivation: Why Function Approximation?

Reinforcement learning is the science of learning to make decisions. An agent can learn a *policy*, a *value function*, and/or a *model* of the environment. The general problem involves taking into account time and consequences: decisions affect the reward, the agent state, and the environment state.

All of the policy, value function, model, and agent state update are functions. We want to learn these from experience. When the state space is too large, we must *approximate*.

Example 1.1 (Large State Spaces)**.** Reinforcement learning can be used to solve large problems:

- Backgammon: $\sim 10^{20}$ states
- Go: $\sim 10^{170}$ states
- Helicopter: continuous state space
- Robots: real-world state spaces

How can we apply our methods for prediction and control to such problems?

When using neural networks to represent these functions, we speak of **deep reinforcement learning**. The term is fairly new ($\pm$7–8 years as of 2021), but the combination of function approximation and RL is fairly old ($\pm$50 years).

2 Value Function Approximation

In previous lectures we mostly considered **lookup tables**: every state s has an entry $v(s)$, or every state-action pair (s, a) has an entry $q(s, a)$.

Problems with large MDPs:

1. There are too many states and/or actions to store in memory.
2. It is too slow to learn the value of each state individually.
3. Individual environment states are often not fully observable.

Definition 2.1 (Value Function Approximation)**.** We estimate the value function with a parametric function approximator:

$$v_{\mathbf{w}}(s) \approx v_\pi(s) \qquad (\text{or } v_*(s)), \tag{1}$$

$$q_{\mathbf{w}}(s, a) \approx q_\pi(s, a) \qquad (\text{or } q_*(s, a)), \tag{2}$$

where $\mathbf{w} \in \mathbb{R}^n$ is a parameter vector. We update $\mathbf{w}$ (e.g. using MC or TD learning) and *generalise* to unseen states.

2.1 Agent State Update

When the environment state is not fully observable ($S_t^{\text{env}} \neq O_t$), we use the **agent state**:

Definition 2.2 (Agent State)**.** The agent state is defined recursively as

$$\mathbf{s}_t = u_\omega(\mathbf{s}_{t-1}, A_{t-1}, O_t), \tag{3}$$

with parameters ω (typically $\omega \in \mathbb{R}^n$). Henceforth, S_t or $\mathbf{s}_t$ denotes the agent state. In the simplest case, $S_t = O_t$.

3 Function Classes

Definition 3.1 (Classes of Function Approximation)**.** The main classes of function approximation are:

1. **Tabular:** a table with an entry for each MDP state.
2. **State aggregation:** partition environment states (or observations) into a discrete set.
3. **Linear function approximation:** with a fixed feature map $\mathbf{x} : \mathcal{S} \to \mathbb{R}^n$ and $v_{\mathbf{w}}(s) = \mathbf{w}^\top \mathbf{x}(s)$. Note: tabular and state aggregation are special cases.
4. **Differentiable (non-linear) function approximation:** $v_{\mathbf{w}}(s)$ is a differentiable function of $\mathbf{w}$ (e.g. a convolutional neural network taking pixels as input). Features are *learnt* rather than fixed.

Remark 3.1 (RL-Specific Challenges)**.** In principle, any function approximator can be used, but RL has specific properties that differentiate it from standard supervised learning:

- Experience is not i.i.d. — successive time-steps are correlated.
- The agent's policy affects the data it receives.
- Regression targets can be non-stationary (due to changing policies, bootstrapping, non-stationary dynamics, and the sheer size of the world).

Remark 3.2 (Choosing a Function Class)**.**

- **Tabular:** good theory but does not scale/generalise.
- **Linear:** reasonably good theory, but requires good hand-crafted features.
- **Non-linear:** less well-understood, but scales well and is less reliant on feature engineering.
- (Deep) neural networks often perform well and remain a popular choice.

4 Gradient-Based Algorithms

4.1 Gradient Descent

Let $J(\mathbf{w})$ be a differentiable function of the parameter vector $\mathbf{w}$.

Definition 4.1 (Gradient)**.** The **gradient** of $J(\mathbf{w})$ is defined as

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \begin{pmatrix} \dfrac{\partial J(\mathbf{w})}{\partial w_1} \\ \vdots \\ \dfrac{\partial J(\mathbf{w})}{\partial w_n} \end{pmatrix}. \tag{4}$$

To minimise $J(\mathbf{w})$, we move $\mathbf{w}$ in the direction of the negative gradient:

$$\Delta \mathbf{w} = -\tfrac{1}{2}\, \alpha\, \nabla_{\mathbf{w}} J(\mathbf{w}), \tag{5}$$

where $\alpha > 0$ is a step-size parameter.

4.2 Stochastic Gradient Descent for Value Approximation

Definition 4.2 (Mean Squared Value Error)**.** The objective (loss) for value function approximation is

$$J(\mathbf{w}) = \mathbb{E}_{S\sim d}\big[(v_\pi(S) - v_{\mathbf{w}}(S))^2\big], \tag{6}$$

where d is a distribution over states (typically induced by the policy and environment dynamics).

Proposition 4.1 (Gradient of MSVE)**.** *The gradient descent update for* $J(\mathbf{w})$ *in* (6) *is*

$$\Delta\mathbf{w} = -\tfrac{1}{2}\,\alpha\,\nabla_{\mathbf{w}} J(\mathbf{w}) = \alpha\,\mathbb{E}_d\big[(v_\pi(S) - v_{\mathbf{w}}(S))\,\nabla_{\mathbf{w}} v_{\mathbf{w}}(S)\big]. \tag{7}$$

Proof. By the chain rule applied to (6):

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \mathbb{E}_d\big[\nabla_{\mathbf{w}}(v_\pi(S) - v_{\mathbf{w}}(S))^2\big] = -2\,\mathbb{E}_d\big[(v_\pi(S) - v_{\mathbf{w}}(S))\,\nabla_{\mathbf{w}} v_{\mathbf{w}}(S)\big],$$

since $v_\pi(S)$ does not depend on $\mathbf{w}$. Multiplying by $-\frac{\alpha}{2}$ yields the result. □

Definition 4.3 (SGD Update for Value Approximation)**. Stochastic gradient descent** (SGD) samples the gradient by replacing $v_\pi(S_t)$ with a sample target (e.g. the Monte Carlo return G_t):

$$\Delta\mathbf{w} = \alpha\,(G_t - v_{\mathbf{w}}(S_t))\,\nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t). \tag{8}$$

Note: G_t is an unbiased sample for $v_\pi(S_t)$. We often write $\nabla v(S_t)$ as shorthand for $\nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t)\big|_{\mathbf{w}=\mathbf{w}_t}$.

5 Linear Function Approximation

5.1 Feature Vectors

Definition 5.1 (Feature Vector)**.** We represent a state by a **feature vector**

$$\mathbf{x}(s) = \begin{pmatrix} x_1(s) \\ \vdots \\ x_n(s) \end{pmatrix} \in \mathbb{R}^n, \tag{9}$$

where $\mathbf{x} : \mathcal{S} \to \mathbb{R}^n$ is a fixed mapping from state (e.g. observation) to features. Shorthand: $\mathbf{x}_t = \mathbf{x}(S_t)$.

Examples of features include: distances of a robot from landmarks, trends in the stock market, and piece/pawn configurations in chess.

Remark 5.1 (Coarse Coding)**. Coarse coding** provides a large feature vector $\mathbf{x}(s)$ by overlapping receptive fields in the state space. The parameter vector $\mathbf{w}$ assigns a value to each feature. This aggregates multiple states, making the resulting feature vector/agent state *non-Markovian* — the common case under function approximation.

5.2 Linear Value Function Approximation

Definition 5.2 (Linear Value Function)**.** The value function is approximated as a linear combination of features:

$$v_{\mathbf{w}}(s) = \mathbf{w}^\top \mathbf{x}(s) = \sum_{j=1}^{n} x_j(s)\,w_j. \tag{10}$$

Proposition 5.1 (Linear SGD Update)**.** *Under the linear parameterisation in Definition 5.2, the objective* (6) *is quadratic in* $\mathbf{w}$*, so SGD converges to the* ***global*** *optimum. Moreover,* $\nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t) = \mathbf{x}(S_t) = \mathbf{x}_t$*, so the update rule simplifies to*

$$\Delta\mathbf{w} = \alpha\,(v_\pi(S_t) - v_{\mathbf{w}}(S_t))\,\mathbf{x}_t. \tag{11}$$

That is: update = step-size × prediction error × feature vector.

Proof. Since $v_{\mathbf{w}}(s) = \mathbf{w}^\top \mathbf{x}(s)$, the gradient with respect to $\mathbf{w}$ is $\nabla_{\mathbf{w}} v_{\mathbf{w}}(s) = \mathbf{x}(s)$. Substituting into Proposition 4.1 yields (11). The loss (6) is $J(\mathbf{w}) = \mathbb{E}[(v_\pi(S) - \mathbf{w}^\top \mathbf{x}(S))^2]$, which is convex quadratic in $\mathbf{w}$, hence SGD converges to the unique global minimum. □

6 Linear Model-Free Prediction

In practice we cannot update towards the true value $v_\pi(s)$ (which is unknown), so we substitute a **target**.

6.1 Monte-Carlo Prediction

Algorithm 6.1 (Linear MC Policy Evaluation)**.** The return G_t is an **unbiased** sample of $v_\pi(S_t)$. We apply supervised learning to the online training data $\{(S_0, G_0), \ldots, (S_t, G_t)\}$:

$$\Delta \mathbf{w}_t = \alpha \left(G_t - v_{\mathbf{w}}(S_t)\right) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t) = \alpha \left(G_t - v_{\mathbf{w}}(S_t)\right) \mathbf{x}_t. \tag{12}$$

Linear MC evaluation converges to the global optimum. Even with non-linear function approximation, MC converges (but perhaps to a local optimum).

6.2 TD Prediction

Algorithm 6.2 (Linear TD(0))**.** The TD target $R_{t+1} + \gamma\, v_{\mathbf{w}}(S_{t+1})$ is a **biased** sample of $v_\pi(S_t)$. We apply supervised learning to $\{(S_0, R_1 + \gamma\, v_{\mathbf{w}}(S_1)),\ \ldots,\ (S_t, R_{t+1} + \gamma\, v_{\mathbf{w}}(S_{t+1}))\}$:

$$\Delta \mathbf{w}_t = \alpha \underbrace{\left(R_{t+1} + \gamma\, v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)\right)}_{=\,\delta_t,\ \text{'TD error'}} \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t) = \alpha\, \delta_t\, \mathbf{x}_t. \tag{13}$$

This is akin to a non-stationary regression problem, but the target depends on our parameters $\mathbf{w}$.

6.3 TD(λ) Prediction

Definition 6.1 (λ-Return)**.** The **λ-return** is defined recursively by

$$G_t^\lambda = R_{t+1} + \gamma\Big((1-\lambda)\, v_{\mathbf{w}}(S_{t+1}) + \lambda\, G_{t+1}^\lambda\Big). \tag{14}$$

Algorithm 6.3 (TD(λ))**.** Using Definition 6.1, the TD(λ) update is

$$\Delta \mathbf{w}_t = \alpha \left(G_t^\lambda - v_{\mathbf{w}}(S_t)\right) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t). \tag{15}$$

7 Control with Value Function Approximation

For control we use **generalised policy iteration**: alternate between approximate policy evaluation ($q_{\mathbf{w}} \approx q_\pi$) and policy improvement (e.g. ε-greedy with respect to $q_{\mathbf{w}}$).

7.1 Action-Value Function Approximation

Definition 7.1 (Action-Value Approximation — Action-In)**.** With **state-action features** $\mathbf{x}(s, a)$, we approximate

$$q_{\mathbf{w}}(s, a) = \mathbf{x}(s, a)^\top \mathbf{w} \approx q_\pi(s, a). \tag{16}$$

The SGD update is

$$\Delta \mathbf{w} = \alpha \left(q_\pi(s, a) - q_{\mathbf{w}}(s, a)\right) \mathbf{x}(s, a). \tag{17}$$

Definition 7.2 (Action-Value Approximation — Action-Out)**.** With **state features** $\mathbf{x}(s) \in \mathbb{R}^n$ and a weight matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$, we produce all action values simultaneously:

$$\mathbf{q}_{\mathbf{W}}(s) = \mathbf{W}\,\mathbf{x}(s) \in \mathbb{R}^m, \qquad q_{\mathbf{W}}(s,a) = \mathbf{q}_{\mathbf{W}}(s)[a] = \mathbf{x}(s)^\top \mathbf{w}_a, \tag{18}$$

where $\mathbf{w}_a = \mathbf{W}_{a\cdot}^\top$ is the a-th row of $\mathbf{W}$. The SGD update modifies only the weights for the selected action:

$$\Delta \mathbf{w}_a = \alpha\,(q_\pi(s,a) - q_{\mathbf{W}}(s,a))\,\mathbf{x}(s), \tag{19}$$

$$\forall\, b \neq a: \quad \Delta \mathbf{w}_b = \mathbf{0}. \tag{20}$$

Equivalently, with the one-hot indicator vector $\mathbf{i}_a \in \mathbb{R}^m$ ($\mathbf{i}_a[a] = 1$, $\mathbf{i}_a[b] = 0$ for $b \neq a$):

$$\Delta \mathbf{W} = \alpha\,(q_\pi(s,a) - q_{\mathbf{W}}(s,a))\,\mathbf{i}_a\,\mathbf{x}(s)^\top. \tag{21}$$

Remark 7.1 (Action-In vs. Action-Out)**.** In the *action-in* formulation, different actions share the same weights $\mathbf{w}$ but use different features $\mathbf{x}(s,a)$. In the *action-out* formulation, the same features $\mathbf{x}(s)$ are shared across actions but each action has its own weight vector. Neither is universally better; for continuous actions, action-in is easier; for small discrete action spaces, action-out is common (e.g. DQN).

Remark 7.2 (SARSA with Function Approximation)**.** SARSA is simply TD applied to state-action pairs, inheriting the same properties as TD for state values. It directly supports policy improvement via ε-greedy, enabling approximate policy iteration.

8 Convergence and Divergence

8.1 Convergence of MC

Theorem 8.1 (MC Fixed Point)**.** *With linear function approximation (Definition 5.2) and suitably decaying step sizes, Monte Carlo converges to*

$$\mathbf{w}_{\mathrm{MC}} = \arg\min_{\mathbf{w}} \mathbb{E}_\pi\big[(G_t - v_{\mathbf{w}}(S_t))^2\big] = \mathbb{E}_\pi[\mathbf{x}_t \mathbf{x}_t^\top]^{-1}\,\mathbb{E}_\pi[G_t\,\mathbf{x}_t]. \tag{22}$$

Proof. At the fixed point, the gradient of the loss vanishes:

$$\begin{aligned}
\nabla_{\mathbf{w}_{\mathrm{MC}}} \mathbb{E}[(G_t - v_{\mathbf{w}_{\mathrm{MC}}}(S_t))^2] &= \mathbb{E}[(G_t - v_{\mathbf{w}_{\mathrm{MC}}}(S_t))\,\mathbf{x}_t] = \mathbf{0} \\
\mathbb{E}[(G_t - \mathbf{x}_t^\top \mathbf{w}_{\mathrm{MC}})\,\mathbf{x}_t] &= \mathbf{0} \\
\mathbb{E}[G_t\,\mathbf{x}_t - \mathbf{x}_t \mathbf{x}_t^\top \mathbf{w}_{\mathrm{MC}}] &= \mathbf{0} \\
\mathbb{E}[\mathbf{x}_t \mathbf{x}_t^\top]\,\mathbf{w}_{\mathrm{MC}} &= \mathbb{E}[G_t\,\mathbf{x}_t] \\
\mathbf{w}_{\mathrm{MC}} &= \mathbb{E}[\mathbf{x}_t \mathbf{x}_t^\top]^{-1}\,\mathbb{E}[G_t\,\mathbf{x}_t].
\end{aligned}$$

□

Remark 8.1. The agent state S_t does not have to be Markov for Theorem 8.1 to hold: the fixed point depends only on observed data (returns and features).

8.2 Convergence of TD

Theorem 8.2 (TD Fixed Point)**.** *With linear function approximation and appropriately decaying step sizes $\alpha_t \to 0$, in a continuing problem with fixed $\gamma < 1$, TD(0) converges to*

$$\mathbf{w}_{\mathrm{TD}} = \mathbb{E}[\mathbf{x}_t(\mathbf{x}_t - \gamma\,\mathbf{x}_{t+1})^\top]^{-1}\,\mathbb{E}[R_{t+1}\,\mathbf{x}_t]. \tag{23}$$

Proof. At convergence, $\mathbb{E}[\Delta \mathbf{w}_{\text{TD}}] = \mathbf{0}$. Assuming α_t does not correlate with $R_{t+1}, \mathbf{x}_t, \mathbf{x}_{t+1}$:

$$\begin{aligned}
\mathbf{0} &= \mathbb{E}[\alpha_t (R_{t+1} + \gamma\, \mathbf{x}_{t+1}^\top \mathbf{w}_{\text{TD}} - \mathbf{x}_t^\top \mathbf{w}_{\text{TD}})\, \mathbf{x}_t] \\
\mathbf{0} &= \mathbb{E}[\alpha_t R_{t+1}\, \mathbf{x}_t] + \mathbb{E}[\alpha_t\, \mathbf{x}_t (\gamma\, \mathbf{x}_{t+1}^\top - \mathbf{x}_t^\top)\, \mathbf{w}_{\text{TD}}] \\
\mathbb{E}[\alpha_t\, \mathbf{x}_t (\mathbf{x}_t - \gamma\, \mathbf{x}_{t+1})^\top]\, \mathbf{w}_{\text{TD}} &= \mathbb{E}[\alpha_t R_{t+1}\, \mathbf{x}_t] \\
\mathbf{w}_{\text{TD}} &= \mathbb{E}[\mathbf{x}_t (\mathbf{x}_t - \gamma\, \mathbf{x}_{t+1})^\top]^{-1}\, \mathbb{E}[R_{t+1}\, \mathbf{x}_t].
\end{aligned}$$

□

Remark 8.2. The TD fixed point (23) generally *differs* from the MC fixed point (22). Typically, the asymptotic MC solution is preferred (smallest prediction error), but TD often converges faster (especially with intermediate $\lambda \in [0, 1]$).

8.3 Value Error Bound for TD

Definition 8.1 (Value Error)**.** The **value error** (VE) is defined as

$$\overline{\text{VE}}(\mathbf{w}) = \|v_\pi - v_{\mathbf{w}}\|_{d_\pi} = \sum_{s \in \mathcal{S}} d_\pi(s)\, (v_\pi(s) - v_{\mathbf{w}}(s))^2, \tag{24}$$

where d_π is the stationary distribution induced by π. The Monte Carlo solution minimises the value error: $\mathbf{w}_{\text{MC}} = \arg\min_{\mathbf{w}} \overline{\text{VE}}(\mathbf{w})$.

Theorem 8.3 (TD Value Error Bound)**.** *The TD fixed point* $\mathbf{w}_{\text{TD}}$ *satisfies*

$$\overline{\text{VE}}(\mathbf{w}_{\text{TD}}) \;\le\; \frac{1}{1-\gamma}\, \overline{\text{VE}}(\mathbf{w}_{\text{MC}}) \;=\; \frac{1}{1-\gamma}\, \min_{\mathbf{w}}\, \overline{\text{VE}}(\mathbf{w}). \tag{25}$$

8.4 TD Is Not a Gradient

Remark 8.3 (Semi-Gradient Property)**.** The TD update

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\, (r + \gamma\, v_{\mathbf{w}}(s') - v_{\mathbf{w}}(s))\, \nabla v_{\mathbf{w}}(s) \tag{26}$$

is **not** a true gradient descent update, because it ignores the dependence of the bootstrapped target $v_{\mathbf{w}}(s')$ on $\mathbf{w}$. It is instead a *stochastic approximation* update — a broader class than SGD. While SGD always converges (under standard conditions), TD does **not** always converge.

8.5 The Deadly Triad: An Example of Divergence

Example 8.1 (Divergence of Off-Policy TD)**.** Consider two states with features $x = 1$ and $x = 2$ (scalar), $r = 0$, and value approximation $v(s) = w\, x(s)$. Thus $v(\text{left}) = w$ and $v(\text{right}) = 2w$. If we update only on the transition from left to right (off-policy), the TD update is

$$\begin{aligned}
w_{t+1} &= w_t + \alpha_t\, (r + \gamma\, v(s') - v(s))\, x(s) \\
&= w_t + \alpha_t\, (0 + \gamma \cdot 2w_t - w_t) \\
&= w_t + \alpha_t\, (2\gamma - 1)\, w_t.
\end{aligned}$$

If $w_t > 0$ and $\gamma > \frac{1}{2}$, then $w_{t+1} > w_t$, so $\lim_{t \to \infty} w_t = \infty$.

Definition 8.2 (Deadly Triad)**.** Algorithms that combine all three of

1. **bootstrapping** (e.g. TD),
2. **off-policy learning**, and
3. **function approximation**

may diverge. This is sometimes called the **deadly triad**.

Example 8.2 (On-Policy Prevents Divergence)**.** Extending Example 8.1 with a third (terminal) state ($v = 0$), and sampling *on-policy* over an episode gives

$$\Delta w = \alpha(0 + 2\gamma w - w) + \alpha(0 + \gamma \cdot 0 - 2w) = \alpha(2\gamma - 3)\, w.$$

Since $2\gamma - 3 < 0$ for all $\gamma \in [0, 1]$, the update is contractive and $w \to 0$ (which is optimal here).

Example 8.3 (Multi-Step Returns and Divergence)**.** Continuing from Example 8.1, using the λ-return (Definition 6.1) with only the left-most state updated:

$$\Delta w = \alpha(2\gamma(1 - \lambda) - 1)\, w.$$

The multiplier is negative when $2\gamma(1 - \lambda) < 1$, i.e. $\lambda > 1 - \frac{1}{2\gamma}$. For $\gamma = 0.9$, we need $\lambda > 4/9 \approx 0.45$.

Example 8.4 (Tabular Prevents Divergence)**.** With tabular features $\mathbf{x} = (1, 0)$ and $\mathbf{x} = (0, 1)$ (one-hot), updating only $v(s) = w[0]$ on the same transition simply regresses $w[0]$ towards $\gamma\, w[1]$. The answer may be sub-optimal (since $w[1]$ is never updated), but no divergence occurs.

8.6 Residual Bellman Updates

Remark 8.4 (Bellman Residual Gradient)**.** The standard TD update $\Delta \mathbf{w}_t = \alpha\, \delta_t\, \nabla v_{\mathbf{w}}(S_t)$ ignores the dependence of $v_{\mathbf{w}}(S_{t+1})$ on $\mathbf{w}$. An alternative is the **Bellman residual gradient** update that minimises $\mathbb{E}[\delta_t^2]$:

$$\Delta \mathbf{w}_t = \alpha\, \delta_t\, \nabla_{\mathbf{w}} \big(v_{\mathbf{w}}(S_t) - \gamma\, v_{\mathbf{w}}(S_{t+1})\big). \tag{27}$$

This tends to work *worse* in practice: Bellman residuals *smooth* values, whereas TD methods *predict*. Smoothed values may lead to suboptimal decisions.

Remark 8.5 (Bellman Error Minimisation)**.** An alternative minimises $\mathbb{E}[\delta_t]^2$ instead:

$$\Delta \mathbf{w}_t = \alpha\, \delta_t\, \nabla_{\mathbf{w}} \big(v_{\mathbf{w}}(S_t) - \gamma\, v_{\mathbf{w}}(S'_{t+1})\big), \tag{28}$$

where S'_{t+1} is a *second independent sample* of the next state (which may differ from S_{t+1}). This double-sample requirement makes online use impractical.

8.7 Summary of Convergence

Table 1: Convergence of prediction algorithms with different function approximation classes. ✓ = converges, ✗ = may diverge.

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	TD	✓	✓	✗
Off-Policy	MC	✓	✓	✓
	TD	✓	✗	✗

Remark 8.6 (Control Convergence)**.** Tabular control algorithms (e.g. Q-learning) can be extended to function approximation (e.g. DQN), but the theory of control with function approximation is not fully developed. **Tracking** (continually adapting the policy) is often preferred to strict convergence.

9 Batch Methods

Online SGD is simple but not sample-efficient. Batch methods seek the best-fitting value function given a set of past experience.

9.1 Least-Squares Temporal Difference (LSTD)

Theorem 9.1 (LSTD Closed-Form Solution)**.** *Under linear function approximation (Definition 5.2), the TD fixed point (Theorem 8.2) can be solved in closed form from finite data. Replacing expectations by empirical averages:*

$$\mathbf{w}_{\text{LSTD}} = \left(\sum_{i=0}^{t} \mathbf{x}_i(\mathbf{x}_i - \gamma\,\mathbf{x}_{i+1})^\top\right)^{-1}\left(\sum_{i=0}^{t} R_{i+1}\,\mathbf{x}_i\right). \tag{29}$$

Proof. By definition of the TD fixed point (23), $\mathbb{E}[\delta_t\,\mathbf{x}_t] = \mathbf{0}$. Replacing the expectation with the empirical mean:

$$\frac{1}{t}\sum_{i=0}^{t}(R_{i+1} + \gamma\,v_{\mathbf{w}}(S_{i+1}) - v_{\mathbf{w}}(S_i))\,\mathbf{x}_i = \mathbf{0}.$$

Substituting $v_{\mathbf{w}}(S) = \mathbf{w}^\top\mathbf{x}(S)$ and solving for $\mathbf{w}$ yields (29). □

Remark 9.1 (Incremental LSTD)**.** Define $\mathbf{A}_t = \sum_{i=0}^{t}\mathbf{x}_i(\mathbf{x}_i - \gamma\,\mathbf{x}_{i+1})^\top$ and $\mathbf{b}_t = \sum_{i=0}^{t} R_{i+1}\,\mathbf{x}_i$, so that $\mathbf{w}_t = \mathbf{A}_t^{-1}\,\mathbf{b}_t$.

- **Naïve** ($O(n^3)$ per step): update $\mathbf{A}_t$ and invert.
- **Sherman–Morrison** ($O(n^2)$ per step): directly maintain $\mathbf{A}_t^{-1}$:

$$\mathbf{A}_{t+1}^{-1} = \mathbf{A}_t^{-1} - \frac{\mathbf{A}_t^{-1}\,\mathbf{x}_t\,(\mathbf{x}_t - \gamma\,\mathbf{x}_{t+1})^\top\mathbf{A}_t^{-1}}{1 + (\mathbf{x}_t - \gamma\,\mathbf{x}_{t+1})^\top\mathbf{A}_t^{-1}\,\mathbf{x}_t}. \tag{30}$$

Both approaches are more expensive per step than TD ($O(n)$).

Remark 9.2 (Extensions of LSTD)**.** LSTD can be extended to multi-step returns (LSTD(λ)), to action values (LSTDQ), and interlaced with policy improvement: **least-squares policy iteration** (LSPI). In the limit, LSTD and incremental TD converge to the same fixed point.

9.2 Experience Replay

Algorithm 9.1 (Experience Replay)**.** Given a buffer of trajectories $\mathcal{D} = \{S_0, A_0, R_1, S_1, \ldots, S_t\}$, repeat:

1. Sample a transition $(S_n, A_n, R_{n+1}, S_{n+1})$ for some $n \le t$.
2. Apply a stochastic gradient update, e.g.

$$\Delta\mathbf{w} = \alpha\,(R_{n+1} + \gamma\,v_{\mathbf{w}}(S_{n+1}) - v_{\mathbf{w}}(S_n))\,\nabla_{\mathbf{w}} v_{\mathbf{w}}(S_n). \tag{31}$$

3. Old data can be re-used.

Caution: the data may be off-policy if the policy has changed since collection.

10 Deep Reinforcement Learning

Many ideas transfer directly when using deep neural networks as function approximators: TD, MC, Double Q-learning, experience replay, etc. Some ideas (e.g. UCB, LSTD/LSTMC) do not easily transfer.

10.1 Neural Q-Learning

Algorithm 10.1 (Online Neural Q-Learning)**.** Components:

- **Neural network:** $O_t \mapsto \mathbf{q_w}$ (action-out).
- **Exploration policy:** $\pi_t = \varepsilon$-greedy$(\mathbf{q}_t)$, $A_t \sim \pi_t$.
- **Weight update:** Q-learning semi-gradient:

$$\Delta \mathbf{w} \propto \Big(R_{t+1} + \gamma \max_a q_{\mathbf{w}}(S_{t+1}, a) - q_{\mathbf{w}}(S_t, A_t)\Big) \nabla_{\mathbf{w}} q_{\mathbf{w}}(S_t, A_t). \tag{32}$$

- **Optimizer:** SGD, RMSProp, Adam, etc.

In practice, the update is implemented via a 'loss':

$$L(\mathbf{w}) = \frac{1}{2}\Big(R_{t+1} + \gamma \overline{\max_a q_{\mathbf{w}}(S_{t+1}, a)} - q_{\mathbf{w}}(S_t, A_t)\Big)^2, \tag{33}$$

where $\bar{\cdot}$ denotes a *stop-gradient* (treating the argument as a constant with respect to $\mathbf{w}$) so that $\nabla_{\mathbf{w}} L$ yields exactly the semi-gradient (32). Note: $L(\mathbf{w})$ is not a true loss; it merely has the desired gradient.

10.2 DQN

Algorithm 10.2 (Deep Q-Network (DQN))**.** DQN (Mnih et al., 2013, 2015) extends neural Q-learning (Algorithm 10.1) with:

- A **replay buffer** storing past transitions $(S_i, A_i, R_{i+1}, S_{i+1})$.
- **Target network** parameters $\mathbf{w}^-$, updated periodically ($\mathbf{w}^- \leftarrow \mathbf{w}$ every C steps, e.g. $C = 10\,000$).
- The Q-learning update uses both replay and the target network:

$$\Delta \mathbf{w} = \Big(R_{i+1} + \gamma \max_a q_{\mathbf{w}^-}(S_{i+1}, a) - q_{\mathbf{w}}(S_i, A_i)\Big) \nabla_{\mathbf{w}} q_{\mathbf{w}}(S_i, A_i). \tag{34}$$

Replay and target networks make RL resemble supervised learning: the training data (from replay) is approximately i.i.d., and the regression targets (from the target network) are approximately stationary. Neither modification is strictly necessary, but both helped stabilise DQN.